

Django CBV (Class-Based Views) -- A to Z Complete Notes

1. Introduction to CBV

****CBV = Class-Based Views****

Django- View :

1. ****Function-Based View (FBV)****
2. ****Class-Based View (CBV)****

FBV Example

```
```python
from django.http import HttpResponse

def home(request):
 return HttpResponse("Hello World")
```
```

CBV Example

```
```python
from django.views import View
from django.http import HttpResponse

class HomeView(View):

 def get(self, request):
 return HttpResponse("Hello World")
```
```

Why use CBV?

CBV- :

- * Code reuse
- * Maintainability
- * Inheritance
- * Mixins
- * Django Generic Views common
- * CRUD operations
- * Authentication permission control
- * HTTP method method

2. Basic CBV

Django- basic CBV `View` class

```
```python
```

```
from django.views import View
```

```
from django.http import HttpResponse
```

```
class MyView(View):
```

```
 def get(self, request):
```

```
 return HttpResponse("This is a GET request")
```

```
 def post(self, request):
```

```
 return HttpResponse("This is a POST request")
```

```
```
```

```
#### HTTP Method    Method
```

| Method | |
|--------------------------|------------------------|
| ----- | ----- |
| <code>`get()`</code> | GET request handle |
| <code>`post()`</code> | POST request handle |
| <code>`put()`</code> | PUT request handle |
| <code>`patch()`</code> | PATCH request handle |
| <code>`delete()`</code> | DELETE request handle |
| <code>`head()`</code> | HEAD request handle |
| <code>`options()`</code> | OPTIONS request handle |
| <code>`trace()`</code> | TRACE request handle |

3. URL- CBV

CBV- URL pattern-

``as_view()``

urls.py

````python`

`from django.urls import path`

`from .views import MyView`

`urlpatterns = [`

`path("myview/", MyView.as_view(), name="myview"),`

`]`

`````

Important

```
```python
```

```
MyView
```

```
```
```

```
**class**
```

```
Django URL resolver **callable**
```

```
:
```

```
```python
```

```
MyView.as_view()
```

```
```
```

class- callable view function-

```
---
```

```
# 4. `as_view()` ?
```

URL- :

```
```python
```

```
path("myview/", MyView.as_view())
```

```
```
```

```
`as_view()` callable function
```

Request Django callable function- execute

Conceptually flow:

```
```text
```

```
Browser
```

URL Dispatcher

MyView.as\_view()

View Callable

MyView instance

setup()

dispatch()

HTTP Method Detect

get() / post() / put() / delete()

Response

Browser

...

### Important Methods

CBV- basic lifecycle- method:

```text

as_view()

setup()

dispatch()

HTTP method handler

...

5. `setup()` Method

`setup()` request URL- positional/keyword arguments instance-

Conceptually:

```
```python
def setup(self, request, *args, **kwargs):
 self.request = request
 self.args = args
 self.kwargs = kwargs
...

```

`setup()` override

override `super()` call :

```
```python
def setup(self, request, *args, **kwargs):
    super().setup(request, *args, **kwargs)

# custom setup logic
...

```

6. `dispatch()` Method

`dispatch()` HTTP method method execute

:

```
```text

```

```
GET get()
POST post()
PUT put()
PATCH patch()
DELETE delete()
...
```

Example:

```
```python
class MyView(View):

    def dispatch(self, request, *args, **kwargs):
        print("Request received")

        return super().dispatch(request, *args, **kwargs)

    def get(self, request, *args, **kwargs):
        return HttpResponse("GET")
...

#### `dispatch()` override ?
```

HTTP method- common logic

:

- * Logging
- * Authentication-related custom check
- * Request preprocessing
- * Common validation

7. ``http_method_not_allowed()``

CBV HTTP method support , Django :

```
```text
```

405 Method Not Allowed

```
```
```

response

Example:

```
```python
```

```
class MyView(View):
```

```
def get(self, request):
```

```
 return HttpResponse("GET request")
```

```
```
```

POST request ``post()``

Django:

```
```text
```

405 Method Not Allowed

```
```
```

```
---
```

8. ``options()`` Method

``OPTIONS`` request handle ``options()`` method

Django- base `View` class handle

application development- manually override

9. Generic Class-Based Views

Django- common built-in Generic CBV

:

```text

ListView

DetailView

CreateView

UpdateView

DeleteView

```

CRUD application

10. ListView

`ListView` object/list

Example:

```python

from django.views.generic import ListView

from .models import Product

```
class ProductListView(ListView):
 model = Product
 template_name = "product_list.html"
 ...
```

Django `Product` model objects template-

Default context variable :

```
```text  
object_list  
...
```

model model-specific context name-
:

```
```python  
class ProductListView(ListView):
 model = Product
 context_object_name = "products"
 template_name = "product_list.html"
 ...
```

Template:

```
```html  
{% for product in products %}  
{{ product.name }}  
{% endfor %}  
...
```

11. ListView- Methods

`ListView`- commonly overridden methods:

```
``text
```

```
get_queryset()
```

```
get_context_data()
```

```
get_paginate_by()
```

```
paginate_queryset()
```

```
render_to_response()
```

```
``
```

```
#### `get_queryset()`
```

Database objects customize

```
``python
```

```
class ProductListView(ListView):
```

```
    model = Product
```

```
    def get_queryset(self):
```

```
        return Product.objects.filter(active=True)
```

```
``
```

12. `get\_context\_data()`

Template- additional data

```
``python
```

```

class ProductListView(ListView):

    model = Product

    def get_context_data(self, **kwargs):

        context = super().get_context_data(**kwargs)

        context["message"] = "Welcome Faruk"

    return context

```

Template:

```

<<html
{{ message }}

```

Important

`super().get_context_data()` , parent class- existing context

13. Pagination with ListView

ListView- pagination

```

<<python

class ProductListView(ListView):

    model = Product

    paginate_by = 10

```

page- object

Template:

```
``html
```

```
{% for product in object_list %}
```

```
{{ product.name }}
```

```
{% endfor %}
```

```
{% if page_obj.has_previous %}
```

Previous

```
{% endif %}
```

```
{% if page_obj.has_next %}
```

Next

```
{% endif %}
```

```
``
```

```
---
```

14. DetailView

```
`DetailView` object
```

```
``python
```

```
from django.views.generic import DetailView
```

```
class ProductDetailView(DetailView):
```

```
    model = Product
```

```
    template_name = "product_detail.html"
```

```
...
```

URL:

```
```python
path(
 "product/",
 ProductDetailView.as_view(),
 name="product-detail",
)
```
```

```
:
```

```
```text
/product/5/
```
```

```
`pk=5` object
```

```
---
```

```
# 15. `get_object()`
```

DetailView- specific object return customize :

```
```python
def get_object(self, queryset=None):
...
```
```

Example:

```

```python
class ProductDetailView(DetailView):
 model = Product

 def get_object(self, queryset=None):
 return Product.objects.get(pk=self.kwargs["pk"])
```

```

Django- default `get\_object()`

16. CreateView

Database- object `CreateView`

```

```python
from django.views.generic import CreateView
from django.urls import reverse_lazy

class ProductCreateView(CreateView):
 model = Product
 fields = ["name", "price", "description"]
 success_url = reverse_lazy("product-list")
```

```

Django automatically form

17. CreateView Form Flow

```
```text
```

GET Request

CreateView

Form

Template- Form

User Submit

POST Request

Form Validation

Valid?

/\

Yes No

save() form\_invalid()

success\_url

```
```
```

```
---
```

18. `form\_valid()`

Form valid `form\_valid()` method call

custom logic

Example:

```
```python
```

```
class ProductCreateView(CreateView):
```

---

```

model = Product

fields = ["name", "price", "description"]

success_url = reverse_lazy("product-list")

def form_valid(self, form):
 form.instance.created_by = self.request.user

 return super().form_valid(form)

```

object save :

```

```python
form.instance.created_by = self.request.user

```

set

19. `form\_invalid()`

Form validation fail :

```

```python
form_invalid()

```

call

Custom error handling override

```

```python
def form_invalid(self, form):

```

```

print(form.errors)

return super().form_invalid(form)
...

---
```

20. Form Handling Methods

Form-based Generic Views- methods:

Method	
<code>`get_form_class()`</code>	Form class return
<code>`get_form()`</code>	Form instance
<code>`get_form_kwargs()`</code>	Form- kwargs
<code>`get_initial()`</code>	Initial values
<code>`form_valid()`</code>	Valid form handle
<code>`form_invalid()`</code>	Invalid form handle
<code>`get_success_url()`</code>	Success-

21. ``get_form_kwargs()``

Form- constructor- data

Example:

```

```python
def get_form_kwargs(self):
kwargs = super().get_form_kwargs()
```

```
kwargs["user"] = self.request.user
```

```
return kwargs
```

```
...
```

custom form-:

```
```python
```

```
def __init__(self, *args, user=None, **kwargs):
```

```
    super().__init__(*args, **kwargs)
```

```
    self.user = user
```

```
...
```

```
---
```

22. UpdateView

Existing object update `UpdateView`

```
```python
```

```
from django.views.generic import UpdateView
```

```
class ProductUpdateView(UpdateView):
```

```
 model = Product
```

```
 fields = ["name", "price", "description"]
```

```
 success_url = reverse_lazy("product-list")
```

```
...
```

URL:

```
```python
```

```
path(
    "product/edit/",
    ProductUpdateView.as_view(),
    name="product-update",
)
---
```

23. DeleteView

Existing object delete `DeleteView`

```
```python
from django.views.generic import DeleteView
```

```
class ProductDeleteView(DeleteView):
 model = Product
 success_url = reverse_lazy("product-list")

```

DeleteView confirmation page

Template:

```
```text
product_confirm_delete.html
---
```

Form submit object delete

24. DeleteView- Methods

DeleteView- commonly used methods:

```
```text
```

```
get_object()
```

```
get_context_data()
```

```
get_success_url()
```

```
post()
```

```
```
```

`delete()` method- Django- DeleteView lifecycle- `post()`- deletion flow

25. Mixins

Mixin reusable class class- additional functionality

Django CBV- Mixins

Common authentication/authorization mixins:

| Mixin | |

| ----- | ----- |

| `LoginRequiredMixin` | Login user- access |

| `UserPassesTestMixin` | Custom condition check |

| `PermissionRequiredMixin` | Django permission check |

Other useful generic view mixins:

```
```text
```

```
FormMixin
```

ModelFormMixin

MultipleObjectMixin

SingleObjectMixin

ContextMixin

...

---

# 26. LoginRequiredMixin

authenticated user- access :

```
```python
```

```
from django.contrib.auth.mixins import LoginRequiredMixin
```

```
from django.views.generic import ListView
```

```
class ProductListView(LoginRequiredMixin, ListView):
```

```
    model = Product
```

```
    template_name = "product_list.html"
```

```
    login_url = "/login/"
```

```
    ...
```

Important

Mixin CBV- inheritance list- :

```
```python
```

```
class ProductListView(LoginRequiredMixin, ListView):
```

```
 ...
```

```
 ...
```

## # 27. UserPassesTestMixin

Custom condition check :

```
```python
from django.contrib.auth.mixins import UserPassesTestMixin

class ProductUpdateView(
    LoginRequiredMixin,
    UserPassesTestMixin,
    UpdateView
):
    model = Product

    def test_func(self):
        return self.request.user == self.get_object().created_by
    ...

user product    update

---
```

28. PermissionRequiredMixin

Django permission check :

```
```python
from django.contrib.auth.mixins import PermissionRequiredMixin

class ProductDeleteView(
 LoginRequiredMixin,
 PermissionRequiredMixin,
```

---

DeleteView

):

model = Product

permission\_required = "shop.delete\_product"

...

user- required permission

---

# 29. Decorators in CBV

Function-based decorator class-

``method_decorator``

````python`

from django.utils.decorators import method_decorator

from django.contrib.auth.decorators import login_required

from django.views import View

from django.http import HttpResponse

@method_decorator(login_required, name="dispatch")

class MyView(View):

def get(self, request):

return HttpResponse("Logged in users only")

...

Recommendation

Authentication- :

```
```python
```

```
LoginRequiredMixin
```

```
```
```

```
natural
```

```
---
```

```
# 30. `method_decorator` Specific Method-
```

```
method- decorator :
```

```
```python
```

```
from django.utils.decorators import method_decorator
```

```
from django.contrib.auth.decorators import login_required
```

```
class MyView(View):
```

```
@method_decorator(login_required)
```

```
def get(self, request):
```

```
return HttpResponse("Protected GET")
```

```
```
```

```
---
```

```
# 31. Request Lifecycle of CBV
```

```
CBV request lifecycle:
```

```
```text
```

```
Browser
```

```
HTTP Request
```

URL Resolver

MyView.as\_view()

Callable View

MyView Instance

setup()

dispatch()

get() / post() / put() / patch() / delete()

Response

Browser

...

Generic View    additional processing

:

```text

ListView

get_queryset()

get_context_data()

render_to_response()

HttpResponse

...

CreateView- :

```
```text
```

CreateView

get\_form()

Form Validation

form\_valid()

OR

form\_invalid()

Response

```
```
```

```
---
```

32. `render_to_response()`

Generic display views- template render response `render_to_response()`

Conceptually:

```
```python
```

```
return self.render_to_response(context)
```

```
```
```

application code- template response customize override

```
---
```

33. CBV Context

Template- data :

```
```python
```

```
get_context_data()
```

```
...
```

Example:

```
```python
```

```
class ProductListView(ListView):
```

```
    model = Product
```

```
    def get_context_data(self, **kwargs):
```

```
        context = super().get_context_data(**kwargs)
```

```
        context["title"] = "Product List"
```

```
        context["total"] = Product.objects.count()
```

```
        return context
```

```
...
```

```
---
```

34. Dynamic Queryset

URL parameter queryset filter

URL:

```
```python
```

```
path(
```

```
 "products/",
```

```
 ProductListView.as_view(),
```

```
 name="product-list",
```

```
)
```

```
...
```

View:

```
```python
```

```
class ProductListView(ListView):
```

```
    model = Product
```

```
    def get_queryset(self):
```

```
        category_id = self.kwargs["category_id"]
```

```
        return Product.objects.filter(
```

```
            category_id=category_id
```

```
        )
```

```
...
```

```
:
```

```
```python
```

```
self.kwargs
```

```
...
```

URL parameter

```

```

```
35. `self.request`
```

CBV- current request :

```
```python
```

```
self.request
```

```
...
```

Example:

```
```python
class ProductListView(ListView):

 def get_queryset(self):
 user = self.request.user

 return Product.objects.filter(
 created_by=user
)
```
```

36. `self.kwargs`

URL- dynamic parameters :

```
```python
self.kwargs
```
```

Example URL:

```
```python
path(
 "product/",
 ProductDetailView.as_view(),
)
```
```

View:

```
```python
def get_object(self, queryset=None):
 pk = self.kwargs["pk"]

 return Product.objects.get(pk=pk)
...

```

# 37. `self.args`

URL- positional arguments :

```
```python
self.args
...

---
```

modern Django application- named URL parameters :

```
```python
self.kwargs
...

```

commonly

---

# 38. `reverse\_lazy()`

CBV- `success\_url` `reverse\_lazy()` common

```
```python
from django.urls import reverse_lazy
```

```
success_url = reverse_lazy("product-list")
```

```
...
```

URL configuration import URL immediately resolve lazy resolve

```
---
```

39. Complete CRUD Example

```
`Product` model
```

```
``python
```

```
from django.views.generic import (
```

```
    ListView,
```

```
    DetailView,
```

```
    CreateView,
```

```
    UpdateView,
```

```
    DeleteView,
```

```
)
```

```
from django.contrib.auth.mixins import (
```

```
    LoginRequiredMixin,
```

```
    UserPassesTestMixin,
```

```
)
```

```
from django.urls import reverse_lazy
```

```
from .models import Product
```

```
...
```

```
## List
```

```
```python
class ProductListView(LoginRequiredMixin, ListView):
 model = Product
 template_name = "product_list.html"
 context_object_name = "products"

 def get_queryset(self):
 return Product.objects.filter(active=True)
```
```

Detail

```
```python
class ProductDetailView(DetailView):
 model = Product
 template_name = "product_detail.html"
```
```

Create

```
```python
class ProductCreateView(LoginRequiredMixin, CreateView):
 model = Product
 fields = ["name", "price", "description"]
 success_url = reverse_lazy("product-list")

 def form_valid(self, form):
 form.instance.created_by = self.request.user

 return super().form_valid(form)
```

```
...
```

```
Update
```

```
```python
class ProductUpdateView(
    LoginRequiredMixin,
    UserPassesTestMixin,
    UpdateView
):
    model = Product
    fields = ["name", "price", "description"]
    success_url = reverse_lazy("product-list")

    def test_func(self):
        return self.request.user == self.get_object().created_by
...

```

```
## Delete
```

```
```python
class ProductDeleteView(
 LoginRequiredMixin,
 UserPassesTestMixin,
 DeleteView
):
 model = Product
 success_url = reverse_lazy("product-list")

 def test_func(self):

```

```
return self.request.user == self.get_object().created_by
```

```
...
```

```

```

#### # 40. CRUD URL Configuration

```
```python
```

```
from django.urls import path
```

```
from .views import (
```

```
    ProductListView,
```

```
    ProductDetailView,
```

```
    ProductCreateView,
```

```
    ProductUpdateView,
```

```
    ProductDeleteView,
```

```
)
```

```
urlpatterns = [
```

```
    path(
```

```
        "products/",
```

```
        ProductListView.as_view(),
```

```
        name="product-list",
```

```
    ),
```

```
    path(
```

```
        "products/",
```

```
        ProductDetailView.as_view(),
```

```
        name="product-detail",
```

```
    ),
```

```

path(
  "products/create/",
  ProductCreateView.as_view(),
  name="product-create",
),

path(
  "products//edit/",
  ProductUpdateView.as_view(),
  name="product-update",
),

path(
  "products//delete/",
  ProductDeleteView.as_view(),
  name="product-delete",
),
]
...

```

41. CBV CRUD Mapping

:

```text

CreateView CREATE

ListView READ (multiple)

DetailView READ (single)

UpdateView UPDATE

DeleteView DELETE

...

:

```text

CRUD

Create Read Update/Delete

Create List Update

Detail Delete

...

42. `get\_queryset()` vs `get\_object()`

`get\_queryset()`

Multiple objects- queryset return

```python

def get\_queryset(self):

return Product.objects.filter(active=True)

...

### `get\_object()`

object return

```

```python
def get_object(self, queryset=None):
    return Product.objects.get(pk=self.kwargs["pk"])
```

```

```

:
```

```

```text
get_queryset()  object
get_object()  object
```

```

```

```

# 43. `get\_context\_data()` vs `get\_queryset()`

```

`get_queryset()`

```

Database data

```

```python
def get_queryset(self):
    return Product.objects.filter(active=True)
```

```

```

`get_context_data()`

```

Template- data

```

```python
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)

```

```
context["message"] = "Hello"
```

```
return context
```

```
...
```

```
:
```

```
```text
```

```
get_queryset()
```

Database data

```
get_context_data()
```

Template context

```
...
```

```

```

#### # 44. Performance Optimization

CBV application automatically optimized

Database query optimize :

```
`select_related`
```

ForeignKey OneToOne relationship- useful

```
```python
```

```
def get_queryset(self):
```

```
return Product.objects.select_related(
```

```
"category"
```

```
)
```

```
...
```

```
### `prefetch_related`
```

ManyToMany reverse relationships- useful

```
```python
```

```
def get_queryset(self):
```

```
 return Product.objects.prefetch_related(
```

```
 "tags"
```

```
)
```

```
```
```

unnecessary database queries

```
---
```

45. CBV AJAX / JSON Response

CBV HTML render

```
`JsonResponse` response
```

```
```python
```

```
from django.http import JsonResponse
```

```
from django.views import View
```

```
class ProductAPIView(View):
```

```
 def get(self, request):
```

```
 return JsonResponse({
```

```
 "message": "Success"
```

```
 })
```

```
```
```

complex REST API Django REST Framework- API views

46. CBV- `super()` ?

Inheritance mixin-based CBV- `super()`

Example:

```
```python
def get_context_data(self, **kwargs):
 context = super().get_context_data(**kwargs)

 context["message"] = "Hello"

 return context
```
```

`super()` , parent class- context

multiple inheritance/mixins- :

```
```python
super()
```
```

47. Common CBV Methods -- Quick Reference

Base `View`

```
```text
```

```
as_view()
setup()
dispatch()
http_method_not_allowed()
options()
...

ListView

```text
get_queryset()
get_context_data()
get_paginate_by()
paginate_queryset()
render_to_response()
...

## DetailView

```text
get_object()
get_queryset()
get_context_data()
render_to_response()
...

CreateView / UpdateView

```text
get_form_class()
```

```

get_form()
get_form_kwargs()
get_initial()
form_valid()
form_invalid()
get_success_url()
get_context_data()
...

```

```

## DeleteView

```

```

``text
get_object()
get_queryset()
get_context_data()
get_success_url()
post()
...

```

```

---
```

```

# 48. FBV vs CBV

```

Feature	FBV	CBV
Structure	Function	Class
Reusability		
Inheritance		
Mixins		
Generic Views		

Simple view		structure
Complex CRUD	Code	Code
HTTP methods	`if request.method`	method
Maintainability	project-	project-

FBV

```
```python
```

```
def product(request):
```

```
 if request.method == "GET":
```

```
 ...
```

```
 elif request.method == "POST":
```

```
 ...
```

```
 ...
```

### CBV

```
```python
```

```
class ProductView(View):
```

```
    def get(self, request):
```

```
        ...
```

```
    def post(self, request):
```

```
        ...
```

```
    ...
```

```
---
```

49. CBV ?

CBV useful :

- * CRUD application
- * logic
- * Inheritance
- * Authentication mixin
- * Permission control
- * Generic View
- * Multiple HTTP methods handle
- * Complex view structure

Simple - view FBV straightforward

50. Common Mistakes

Mistake 1

```
```python
path("products/", ProductListView)
```
```

:

```
```python
path(
 "products/",
 ProductListView.as_view(),
)
```
```

Mistake 2

``get_queryset()`- `super()` override`

````python`

`def get_queryset(self):`

`return Product.objects.all()`

`````

, parent queryset- filtering/customization :

````python`

`def get_queryset(self):`

`queryset = super().get_queryset()`

`return queryset.filter(active=True)`

`````

Mistake 3

``get_context_data()`- parent context`

````python`

`def get_context_data(self, **kwargs):`

`return {`

`"message": "Hello"`

`}`

```
...
```

parent context

```
:
```

```
```python
```

```
def get_context_data(self, **kwargs):
```

```
    context = super().get_context_data(**kwargs)
```

```
    context["message"] = "Hello"
```

```
    return context
```

```
...
```

```
---
```

Mistake 4

Create/Update View- `success\_url`

```
```python
```

```
success_url = reverse_lazy("product-list")
```

```
...
```

```
:
```

```
```python
```

```
def get_success_url(self):
```

```
    return reverse("product-detail", kwargs={"pk": self.object.pk})
```

```
...
```

```
---
```

51. Interview Important Questions

Q1. CBV ?

CBV Django View Python class-

Q2. `as\_view()` ?

Class-based view- URL dispatcher- callable view-

Q3. `dispatch()` ?

HTTP request method appropriate handler method

Q4. `get\_queryset()` ?

Database queryset

Q5. `get\_object()` ?

object retrieve

Q6. `get\_context\_data()` ?

Template- context data customize

Q7. `form\_valid()` call ?

Form validation

Q8. `form\_invalid()` call ?

Form validation

Q9. `LoginRequiredMixin` ?

authenticated user- CBV access

Q10. `UserPassesTestMixin`?

Custom condition- access control

Q11. `PermissionRequiredMixin`?

Django permission- access control

Q12. `reverse\_lazy()` CBV- ?

URL configuration- lazy URL reversing , class attribute `success\_url`-

52. CBV Master Flow

flow:

```text

Browser

URL Resolver

MyView.as\_view()

View Callable

Class Instance

setup()

dispatch()

get() post() delete()

Response

Browser

...

Generic CRUD:

```text

Generic CBV

ListView DetailView Form Views

get_queryset() get_object() get_form()

get_context_data() form_valid()

HttpResponse

...

53. Final A-to-Z Summary

Django CBV- core concepts:

1. **CBV** = Class-Based View
2. **View** = Base class
3. **as_view()** = Class- callable view-
4. **setup()** = Request arguments initialize

5. `**`dispatch()`**` = HTTP method handler
6. `**`get()`**` = GET request handle
7. `**`post()`**` = POST request handle
8. `**`put()`**` = PUT request handle
9. `**`patch()`**` = PATCH request handle
10. `**`delete()`**` = DELETE request handle
11. `**`ListView`**` = Multiple objects
12. `**`DetailView`**` = Single object
13. `**`CreateView`**` = object
14. `**`UpdateView`**` = Existing object update
15. `**`DeleteView`**` = Object delete
16. `**`get_queryset()`**` = Queryset customize
17. `**`get_object()`**` = Single object retrieve
18. `**`get_context_data()`**` = Template context customize
19. `**`get_form()`**` = Form instance
20. `**`form_valid()`**` = Valid form handle
21. `**`form_invalid()`**` = Invalid form handle
22. `**`get_success_url()`**` = Success- redirect location
23. `**`LoginRequiredMixin`**` = Login protection
24. `**`UserPassesTestMixin`**` = Custom access condition
25. `**`PermissionRequiredMixin`**` = Permission-based access
26. `**`method_decorator`**` = CBV- function decorator
27. `**`select_related()`**` = ForeignKey/OneToOne query optimization
28. `**`prefetch_related()`**` = ManyToMany/reverse relation optimization
29. `**`super()`**` = Parent/Mixin behavior
30. `**`reverse_lazy()`**` = Lazy URL reversing
31. `**`JsonResponse`**` = JSON response

32. ****CRUD**** = Create + Read + Update + Delete

One-Line Memory Trick

```text

View

as\_view()

setup()

dispatch()

get()/post()/put()/patch()/delete()

Generic View Logic

Response

```

CRUD :

```text

Create CreateView

Read ListView + DetailView

Update UpdateView

Delete DeleteView

```

**** Django CBV- core structure****